

# Handling Real 3D Medical Volumetric Data

*A complete walkthrough of one project: loading, exploring, and segmenting 3D brain MRI volumes from raw NIfTI files through a trained 3D U-Net. Written to be read start to finish, with the reasoning behind each decision and the code that implements it.*

## Contents

Preface

### Part I. The Data and the Coordinate System

1. What This Project Asks
  - 1.1 The question
  - 1.2 Why this is harder than image classification
  - 1.3 What BraTS provides
  - 1.4 The shape of the deliverable
2. The NIfTI Format
  - 2.1 What NIfTI is
  - 2.2 Loading a NIfTI file
  - 2.3 Voxel spacing and the header
3. The Affine Transformation Matrix
  - 3.1 The problem
  - 3.2 The solution: the 4x4 affine matrix
  - 3.3 Voxel-to-world and world-to-voxel
  - 3.4 Anatomical orientation codes
  - 3.5 Why all of this matters for the model

### Part II. The Data Pipeline

4. The MONAI Transform System
  - 4.1 Dictionary transforms
  - 4.2 The full preprocessing pipeline
  - 4.3 Training-only augmentation
  - 4.4 Patch-based training
  - 4.5 Why validation is different
5. Dataset and DataLoaders
  - 5.1 The dataset.json manifest
  - 5.2 Train/validation split
  - 5.3 The DataLoader

### Part III. The Model

6. The U-Net Architecture
  - 6.1 Why U-Net
  - 6.2 The 3D architecture
  - 6.3 The building blocks
  - 6.4 Forward pass shape trace
7. The Loss Function
  - 7.1 The class imbalance problem
  - 7.2 Dice Loss
  - 7.3 DiceCELoss: combined objective
8. Training
  - 8.1 Optimizer and learning rate schedule
  - 8.2 Saving the best checkpoint, not the last
  - 8.3 The training loop
9. Handling Class Imbalance in the Metric

- 9.1 Why overall voxel accuracy is misleading
- 9.2 The Dice Metric

## **Part IV. Inference and Visualization**

- 10. Sliding-Window Inference
  - 10.1 The memory problem
  - 10.2 Sliding-window inference
  - 10.3 Post-processing
- 11. The BraTS Evaluation Hierarchy
  - 11.1 Why not just Dice per label
  - 11.2 Computing the metrics
- 12. Publication-Quality Static Visualization
  - 12.1 Why static
  - 12.2 Multi-Planar Reconstruction (MPR)
  - 12.3 Volumetric Slice Montage
  - 12.4 The Affine Coordinate Space Diagram
  - 12.5 Static 3D Tumor Surface Mesh
  - 12.6 The Segmentation Evaluation Dashboard
- 13. Maximum Intensity Projection
- 14. Where This Leads
- Appendix A. Environment and Reproduction
- Appendix B. Pitfalls Encountered
- Appendix C. Glossary



## Preface

---

Most deep learning tutorials show you a model that works on images. Medical imaging is different in three ways that matter before the first line of training code runs.

The data is three-dimensional. A brain MRI is not a collection of photographs; it is a spatial volume of measurements, and the machine that produced it assigned each measurement a physical location in the patient's body. Handling that correctly requires understanding a coordinate system that has no equivalent in ordinary computer vision.

The data comes in four channels simultaneously, not one. Four separate MRI acquisition sequences cover the same brain, each sensitive to different tissue contrasts. Fusing them is not optional; each modality sees something the others miss.

The task is segmentation, not classification. The output is not a label for the whole image but a label for every voxel: three thousand times as many decisions per forward pass as classification, with a loss function and evaluation metric calibrated specifically for the severe class imbalance that results.

This document walks through all three of those problems, in the order the reasoning happens rather than the order the code runs. It has four parts.

**Part I** builds the vocabulary. What a NIfTI volume is, how the affine matrix maps voxel indices to physical space, and why the four MRI modalities look different from each other.

**Part II** develops the preprocessing and data pipeline. How MONAI's dictionary transform system works, why the training pipeline extracts patches while validation operates on whole volumes, and what the loss function is actually measuring.

**Part III** describes the model. The 3D U-Net architecture, how skip connections move information around in a way that matters for segmentation, and how sliding-window inference avoids running out of memory on full-size volumes.

**Part IV** covers visualization and evaluation. How to read orthogonal slice views, what the BraTS evaluation metrics actually measure, and what the static publication-quality figures produced by this project's visualization module show.

Code listings are real excerpts from the project's source files, lightly trimmed for reading.

---

## **Part I. The Data and the Coordinate System**

# 1. What This Project Asks

---

## 1.1 The question

Give a computer a stack of 3D MRI scans of a patient's brain, across four acquisition protocols, and ask it to locate and classify every voxel of tumor. Produce a label for each of three tumor tissue compartments: the necrotic core, the surrounding edema, and the actively growing enhancing rim.

This is a well-defined clinical problem with a well-defined benchmark dataset and well-defined evaluation metrics. The Brain Tumor Segmentation challenge (BraTS) has run since 2012 and its dataset, packaged as MSD Task01\_BrainTumour, is what this project uses.

## 1.2 Why this is harder than image classification

The difficulty is not primarily the deep learning. It is the data representation.

A brain MRI volume is a three-dimensional array of measurements taken at specific physical locations in the scanner's coordinate frame. The relationship between array indices and physical coordinates is not obvious and is not constant across volumes. Different volumes from different scanners can have different resolutions, different field-of-view sizes, and different orientations of the patient relative to the scanner axes.

A model trained on one such distribution and evaluated on another will appear to work but be learning the wrong thing. Getting this right requires understanding the affine matrix, which is what Chapter 3 is about.

## 1.3 What BraTS provides

The BraTS dataset distributed by the Medical Segmentation Decathlon ships as `.nii.gz` files, one compressed NIfTI archive per subject per split. Each training subject has two files: a four-channel image volume and a three-class label volume.

The four image channels are:

- **Channel 0: FLAIR** (Fluid-Attenuated Inversion Recovery). Suppresses the bright signal from cerebrospinal fluid, making the white signal of peritumoral edema stand out against dark background.
- **Channel 1: T1w** (T1-weighted native). Standard anatomical contrast; gray matter is darker than white matter. Used for structural reference and ventricle delineation.
- **Channel 2: T1ce** (T1-weighted contrast-enhanced). A gadolinium contrast agent is injected intravenously. Where the blood-brain barrier has been disrupted by the growing tumor, the agent leaks into tissue and creates a bright signal. This directly marks the active, vascularized tumor core.
- **Channel 3: T2w** (T2-weighted). Sensitive to water content. Whole tumor region and edema appear bright. Gives the broadest view of lesion extent.

No single modality shows the complete picture. FLAIR best delineates edema; T1ce best delineates the enhancing core; T2 gives the broadest lesion boundary. Multi-modal fusion is not an architectural choice; it is required by the biology.

The label volume has four possible values at each voxel: 0 (background), 1 (necrotic and non-enhancing tumor core, NCR), 2 (peritumoral edema, ED), and 3 (GD-enhancing tumor, ET). The hierarchy of BraTS evaluation sub-regions is derived from combinations of these labels and is described in Chapter 14.

## 1.4 The shape of the deliverable

This project produces a trained segmentation model and a visualization module. Specifically:

- A MONAI preprocessing pipeline that standardizes volumes to 1 mm isotropic resolution in RAS orientation.
- A custom 3D U-Net trained with DiceCELoss.

- A sliding-window inference procedure that evaluates whole volumes without requiring GPU memory proportional to the full volume size.
  - A static publication-quality visualization module covering orthogonal MPR views, volumetric slice montages, 3D tumor mesh reconstructions, and segmentation evaluation dashboards.
-

## 2. The NIfTI Format

---

### 2.1 What NIfTI is

NIfTI (Neuroimaging Informatics Technology Initiative) is the standard file format for neuroimaging data. A `.nii.gz` file is a gzip-compressed NIfTI-1 archive containing two parts: a 348-byte header and the raw voxel data array.

The header carries the metadata that makes the file scientifically meaningful: the shape of the array, the physical size of each voxel, the data type, and the affine transformation matrix. The voxel array itself is just numbers; the header is what turns those numbers into measurements with physical units.

### 2.2 Loading a NIfTI file

```
import nibabel as nib

img_obj = nib.load("BRATS_095.nii.gz")
data = img_obj.get_fdata()      # float64 array, shape (240, 240, 155, 4)
affine = img_obj.affine         # 4x4 float64 matrix
header = img_obj.header
```

`get_fdata()` decompresses and type-casts the raw voxels to 64-bit float. The BraTS volumes arrive as shape (240, 240, 155, 4): 240 voxels in the first two spatial dimensions, 155 in the third, and 4 channels last.

MONAI's `LoadImaged` transform performs the same operation, but as part of a dictionary pipeline where image and label are loaded together:

```
from monai.transforms import LoadImaged, EnsureChannelFirstd

loader = LoadImaged(keys=["image", "label"], image_only=False)
sample = loader({"image": img_path, "label": lbl_path})
# sample["image"].shape: (240, 240, 155, 4) or (4, 240, 240, 155) after EnsureChannelFirstd
```

### 2.3 Voxel spacing and the header

The header stores the physical size of each voxel in millimeters, called the *zooms* or pixel dimensions:

```
zooms = header.get_zooms()
# (1.0, 1.0, 1.0, 1.0) for a 1 mm isotropic BraTS volume
```

A voxel at index (i, j, k) covers a region of physical space measuring  $dx * dy * dz$  cubic millimeters, where (dx, dy, dz) are the spatial zooms. Knowing this is necessary to compute volumes in  $cm^3$  and to resample correctly between different resolutions.

---



### 3. The Affine Transformation Matrix

---

This is the idea that shapes every coordinate-related decision in the project, so it earns its own chapter.

#### 3.1 The problem

A 3D array has no inherent location in space. Index (0, 0, 0) could correspond to any physical point; the step from one index to the next could correspond to any physical distance. The same brain scanned on two different machines can produce arrays with different shapes, different resolutions, and different orientations of the scanner axes relative to the patient's head.

A model trained on one such distribution will see images where, say, the nose points in the positive-x direction. If evaluated on another distribution where the nose points in the negative-x direction, the spatial relationships between features will be mirror-reversed. The model will be confidently wrong in ways that are invisible without inspecting the coordinate system.

#### 3.2 The solution: the 4x4 affine matrix

Every NIfTI file contains a 4x4 affine transformation matrix M that maps from discrete voxel indices (i, j, k) to continuous physical scanner coordinates (x, y, z) in millimeters:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} M00 & M01 & M02 & M03 \\ M10 & M11 & M12 & M13 \\ M20 & M21 & M22 & M23 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} i \\ j \\ k \\ 1 \end{bmatrix}$$

The top-left 3x3 submatrix encodes both rotation (orientation of the scanner axes) and scaling (voxel spacing). The rightmost column encodes translation (the physical location of voxel index (0, 0, 0)).

#### 3.3 Voxel-to-world and world-to-voxel

Two utility functions implement these coordinate transformations:

```
def voxel_to_world(voxel_coords: np.ndarray, affine: np.ndarray) -> np.ndarray:
    """Transform voxel indices (i, j, k) to physical scanner coordinates (x, y, z) in mm."""
    voxel_coords = np.asarray(voxel_coords)
    is_single = (voxel_coords.ndim == 1)
    if is_single:
        voxel_coords = voxel_coords[np.newaxis, :]
    homo = np.hstack([voxel_coords, np.ones((voxel_coords.shape[0], 1))])
    world = homo @ affine.T
    return world[0, :3] if is_single else world[:, :3]

def world_to_voxel(world_coords: np.ndarray, affine: np.ndarray) -> np.ndarray:
    """Transform physical coordinates (x, y, z) in mm to nearest voxel indices (i, j, k)."""
    inv_affine = np.linalg.inv(affine)
    homo = np.hstack([world_coords, np.ones((world_coords.shape[0], 1))])
    voxels = homo @ inv_affine.T
    return np.round(voxels[:, :3]).astype(int)
```

The homogeneous coordinate trick (appending 1 to each point) allows translation to be applied as a matrix multiplication, which is why 4x4 matrices rather than 3x3 are used.

#### 3.4 Anatomical orientation codes

From the affine matrix, nibabel can derive which direction each array axis points in anatomical space:

```
import nibabel as nib

orientation = nib.aff2axcodes(affine)
# ('R', 'A', 'S') means axis 0 points Right, axis 1 Anterior, axis 2 Superior
```

**RAS** (Right-Anterior-Superior) is the neurological convention: the x-axis points toward the patient's right ear, y toward the nose, z toward the top of the head. **LPS** (Left-Posterior-Superior) is the radiological convention used by DICOM.

The MONAI `Orientationd` transform reorients any incoming volume to RAS regardless of what the scanner produced. Without this, two volumes from different scanners could be in different orientations and the model would see anatomically opposite directions as equivalent.

### 3.5 Why all of this matters for the model

`Orientationd` and `Spacingd` together guarantee that every volume entering the model represents the same physical coordinate system at the same resolution. Without them, the spatial relationship between features would shift arbitrarily between subjects. The model would have to learn to be invariant to something that the preprocessing can simply remove.

---

## **Part II. The Data Pipeline**

## 4. The MONAI Transform System

---

### 4.1 Dictionary transforms

MONAI's transform system processes samples as Python dictionaries rather than bare arrays. Every transform takes a dict in and returns a dict out. Keys identify which arrays to operate on:

```
sample = {"image": image_array, "label": label_array}
transform = Orientationd(keys=["image", "label"], axcodes="RAS")
result = transform(sample)
# result["image"] and result["label"] are both reoriented
```

The dictionary approach means that spatial transforms like `Orientationd` and `Spacingd` automatically apply the same geometric operation to both the image and its corresponding label. There is no way to accidentally apply a flip to the image but not the label.

### 4.2 The full preprocessing pipeline

The base transforms applied to both training and validation data are:

```
base_transforms = [
    LoadImaged(keys=["image", "label"], image_only=False),
    EnsureChannelFirstd(keys=["image", "label"]),
    Orientationd(keys=["image", "label"], axcodes="RAS"),
    Spacingd(
        keys=["image", "label"],
        pixdim=(1.0, 1.0, 1.0),
        mode=("bilinear", "nearest"),
    ),
    NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
    EnsureTyped(keys=["image", "label"], dtype=torch.float32),
]
```

Each step matters:

**EnsureChannelFirstd** moves the channel axis to the front, giving (C, D, H, W) layout, which is what PyTorch convolutions expect.

**Orientationd(axcodes="RAS")** reorients both image and label to the RAS anatomical convention, as explained in Chapter 3.

**Spacingd(pixdim=(1.0, 1.0, 1.0))** resamples both arrays to 1mm isotropic voxel spacing. The image uses bilinear interpolation (smooth, good for intensity data). The label uses nearest-neighbor interpolation (integer-valued class labels must not be blurred to fractional values).

**NormalizeIntensityd(nonzero=True, channel\_wise=True)** computes the mean and standard deviation of each channel independently, considering only the nonzero voxels. The zero background in brain MRI represents air outside the skull, not meaningful signal. Including it in the normalization statistics would distort the mean and standard deviation. **channel\_wise=True** keeps the four modalities on separate intensity scales, since they were acquired with different physical contrast mechanisms.

### 4.3 Training-only augmentation

Beyond the base transforms, training applies:

```

RandCropByPosNegLabeld(
    keys=["image", "label"],
    label_key="label",
    spatial_size=(96, 96, 96),
    pos=2,
    neg=1,
    num_samples=2,
    image_threshold=0,
),
RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),
RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=2),
RandRotated(keys=["image", "label"], range_x=0.2, range_y=0.2, range_z=0.2,
            mode=("bilinear", "nearest"), prob=0.3),
RandGaussianNoised(keys="image", prob=0.2, std=0.1),

```

#### 4.4 Patch-based training

A whole-volume BraTS scan at 1mm isotropic resolution is roughly (4, 240, 240, 155) after preprocessing. Storing one such volume in GPU memory during a forward pass requires hundreds of megabytes; a batch of them does not fit.

The solution is to train on patches: 96x96x96 sub-volumes extracted from the full volume. `RandCropByPosNegLabeld` extracts patches with a deliberate spatial bias:

- **Positive crops:** centered on a foreground (tumor) voxel, with probability proportional to  $\text{pos} / (\text{pos} + \text{neg}) = 2/3$ .
- **Negative crops:** centered on a background voxel, with probability  $1/3$ .

The bias toward tumor-containing patches is necessary because tumor voxels are a small fraction of the total. Without it, most patches would contain no tumor at all, and the model would see essentially no positive examples.

`num_samples=2` means two patches are extracted per volume per epoch, so the effective dataset size is multiplied by 2 at no additional storage cost.

#### 4.5 Why validation is different

Validation does not use patches. Patches would introduce randomness into the evaluation: the score would vary depending on which patches happened to be sampled, and the same model would produce a different number each run.

Instead, validation uses `sliding_window_inference` on the full volume (described in Chapter 10). The validation transform pipeline is just the base transforms plus type casting, with no cropping or augmentation.

---

## 5. Dataset and DataLoaders

---

### 5.1 The dataset.json manifest

The MSD dataset ships a `dataset.json` file that lists all training and test subject paths, along with metadata:

```
{
  "name": "BrainTumour",
  "modality": {"0": "FLAIR", "1": "T1", "2": "T1ce", "3": "T2"},
  "labels": {"0": "background", "1": "edema", "2": "non-enhancing tumor",
             "3": "enhancing tumor"},
  "training": [
    {"image": "./imagesTr/BRATS_001.nii.gz",
      "label": "./labelsTr/BRATS_001.nii.gz"},
    ...
  ]
}
```

The dataset loader parses this file and constructs lists of dictionaries, one per subject, for MONAI's `Dataset` class:

```
formatted_data = [
    {
        "image": str((data_dir / item["image"]).replace("./", "").resolve()),
        "label": str((data_dir / item["label"]).replace("./", "").resolve()),
        "id": Path(img_path).stem.replace(".nii", "")
    }
    for item in raw_training
]
```

### 5.2 Train/validation split

The training subjects are split into training and validation at load time using scikit-learn's `train_test_split`:

```
train_files, val_files = train_test_split(
    formatted_data, test_size=0.2, random_state=42, shuffle=True
)
```

An 80/20 split is used. The random seed is fixed so the same subjects always go into training and validation. MSD does not ship a pre-defined validation set, so this split must be constructed from the training data.

### 5.3 The DataLoader

MONAI's `Dataset` wraps the list of file paths and applies the transform pipeline on each access. The MONAI `DataLoader` wraps the dataset and produces mini-batches:

```
train_ds = Dataset(data=train_files, transform=train_transforms)
train_loader = DataLoader(
    train_ds, batch_size=2, shuffle=True,
    pin_memory=torch.cuda.is_available()
)
```

During training, the `DataLoader` provides batches of shape (2, 4, 96, 96, 96): two subjects, four modality channels, 96x96x96 spatial patch. Actually, since `num_samples_per_volume=2`, the training

transform produces 2 patches per volume, and the batch collation sees them as 2 independent training examples.

---

## **Part III. The Model**



## 6. The U-Net Architecture

---

### 6.1 Why U-Net

U-Net is the dominant architecture for biomedical image segmentation. It was originally proposed for 2D histology images but extends directly to 3D volumetric data. Its essential properties are:

First, it produces a dense output: a label for every voxel in the input, not a single label for the whole image. This requires the network to maintain spatial resolution information through to the output.

Second, it uses encoder-decoder structure with skip connections: the encoder compresses spatial resolution while increasing feature depth; the decoder recovers spatial resolution; skip connections copy feature maps from the encoder directly to the corresponding decoder level, bypassing the bottleneck.

The skip connections are the critical element. Without them, the decoder must reconstruct precise spatial structure from a highly compressed representation. With them, fine spatial detail is never lost; it is simply copied around the compression.

### 6.2 The 3D architecture

The custom 3D U-Net in `src/model.py` is a direct extension to three spatial dimensions:

```
class CustomUNet3D(nn.Module):
    def __init__(self, in_channels=4, out_channels=4,
                  features=(16, 32, 64, 128)):
        super().__init__()
        self.inc = DoubleConv3D(in_channels, features[0], residual=False)
        self.down1 = DownBlock3D(features[0], features[1])
        self.down2 = DownBlock3D(features[1], features[2])
        self.down3 = DownBlock3D(features[2], features[3])
        self.up1 = UpBlock3D(features[3], features[2])
        self.up2 = UpBlock3D(features[2], features[1])
        self.up3 = UpBlock3D(features[1], features[0])
        self.outc = nn.Conv3d(features[0], out_channels, kernel_size=1)
```

The encoder path (`inc`, `down1`, `down2`, `down3`) halves the spatial resolution at each `DownBlock3D` while doubling the number of feature channels. A 96x96x96 patch at the input becomes 12x12x12 at the bottleneck.

The decoder path (`up1`, `up2`, `up3`) inverts this: each `UpBlock3D` doubles the spatial resolution via transposed convolution and concatenates the corresponding encoder skip connection.

### 6.3 The building blocks

**DoubleConv3D** applies two sequential 3D convolutions, each followed by Instance Normalization and LeakyReLU:

```
self.conv = nn.Sequential(
    nn.Conv3d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
    nn.InstanceNorm3d(out_ch),
    nn.LeakyReLU(negative_slope=0.01, inplace=True),
    nn.Conv3d(out_ch, out_ch, kernel_size=3, padding=1, bias=False),
    nn.InstanceNorm3d(out_ch),
    nn.LeakyReLU(negative_slope=0.01, inplace=True),
)
```

**Instance Normalization** rather than Batch Normalization is the standard choice for medical image segmentation. Batch Norm normalizes across the batch dimension; with the small batch sizes (1-2 volumes) that 3D medical segmentation requires, the batch statistics are too noisy to be useful. Instance Norm normalizes each sample independently, which is stable regardless of batch size.

**DownBlock3D** applies MaxPool3d with stride 2 before the double convolution. Max pooling halves the spatial resolution and discards the weaker activations at each spatial location.

**UpBlock3D** uses ConvTranspose3d to double the spatial resolution, then concatenates the skip connection and applies a double convolution to fuse the two feature streams:

```
def forward(self, x: torch.Tensor, skip: torch.Tensor) -> torch.Tensor:
    x = self.up(x) # ConvTranspose3d: double spatial size
    x = torch.cat([skip, x], dim=1) # Concatenate skip connection
    return self.conv(x) # DoubleConv3D to fuse
```

## 6.4 Forward pass shape trace

For a 96x96x96 input with 4 channels:

| Stage       | Shape                | Notes                      |
|-------------|----------------------|----------------------------|
| Input       | (B, 4, 96, 96, 96)   | 4 MRI modalities           |
| After inc   | (B, 16, 96, 96, 96)  | First double conv          |
| After down1 | (B, 32, 48, 48, 48)  | MaxPool + conv             |
| After down2 | (B, 64, 24, 24, 24)  | MaxPool + conv             |
| After down3 | (B, 128, 12, 12, 12) | Bottleneck                 |
| After up1   | (B, 64, 24, 24, 24)  | Upsample + skip from down2 |
| After up2   | (B, 32, 48, 48, 48)  | Upsample + skip from down1 |
| After up3   | (B, 16, 96, 96, 96)  | Upsample + skip from inc   |
| Output      | (B, 4, 96, 96, 96)   | 4 class logits per voxel   |

The output has the same spatial dimensions as the input, with one logit per class per voxel.

## 7. The Loss Function

---

### 7.1 The class imbalance problem

Tumor voxels are a small fraction of the total brain volume. In BraTS data, background voxels can outnumber tumor voxels by 50:1 or more. A model that predicts background everywhere would score 98% accuracy while being clinically useless.

Standard cross-entropy loss treats all voxels equally. Applied to this distribution, it produces a gradient dominated by the majority class, and the model learns quickly that predicting background is almost always correct.

### 7.2 Dice Loss

The Dice coefficient, borrowed from set theory, measures the overlap between the predicted and ground-truth segmentations:

$$\text{Dice}(P, G) = 2 * |P \cap G| / (|P| + |G|)$$

where  $|P|$  is the number of predicted tumor voxels,  $|G|$  the number of ground-truth tumor voxels, and  $|P \cap G|$  the number that are both. It ranges from 0 (no overlap) to 1 (perfect overlap).

Dice Loss is  $1 - \text{Dice}$ , so minimizing it maximizes overlap. Its key property is that it is computed separately per class and then averaged: a class with 100 true tumor voxels and 50 predictions gets the same weight in the gradient as a class with 10,000 true voxels and 5,000 predictions. The loss is normalized by the number of voxels in each class, not the total volume.

### 7.3 DiceCELoss: combined objective

The training loss function combines Dice Loss with standard cross-entropy:

```
from monai.losses import DiceCELoss

loss_function = DiceCELoss(to_onehot_y=True, softmax=True)
```

`to_onehot_y=True` tells MONAI to convert the integer label tensor to one-hot encoding before computing the loss. `softmax=True` applies softmax to the model's raw logits before computing Dice, since Dice requires probabilities that sum to 1.

The combination is useful because Dice Loss alone can be unstable early in training when predictions are near-random; cross-entropy provides a smooth gradient everywhere and stabilizes optimization.

---

## 8. Training

---

### 8.1 Optimizer and learning rate schedule

```
optimizer = AdamW(model.parameters(), lr=2e-4, weight_decay=1e-5)
scheduler = CosineAnnealingLR(optimizer, T_max=epochs)
```

AdamW is Adam with decoupled weight decay: the weight decay term is applied directly to the parameters rather than being folded into the gradient update. This is the standard choice for vision tasks.

Cosine annealing decays the learning rate smoothly from the initial value to near zero over the training run. Early epochs take larger steps to find a good basin; later epochs take smaller steps to settle into the minimum without bouncing. The schedule matches the one used in the reference MONAI training recipes.

### 8.2 Saving the best checkpoint, not the last

```
if val_dice > best_metric:
    best_metric = val_dice
    best_epoch = epoch
    torch.save(model.state_dict(), output_dir / "best_metric_model.pth")
```

With a cosine schedule, the model at the final epoch is usually near its best, but not always. Using the last checkpoint would be convenient but wrong: the model that actually produced the best validation performance might have been from five epochs earlier.

The checkpoint is saved only when a new best is found. The comparison at the end uses `model.load_state_dict(best["state"])` to restore the correct weights.

### 8.3 The training loop

```
for epoch in range(1, epochs + 1):
    train_loss = train_epoch(model, train_loader, optimizer, loss_function, device)
    lr_scheduler.step()

    if epoch % val_interval == 0:
        val_dice = validate_epoch(
            model, val_loader, roi_size, sw_batch_size,
            dice_metric, post_trans, device
        )
        if val_dice > best_metric:
            best_metric = val_dice
            torch.save(model.state_dict(), output_dir / "best_metric_model.pth")
```

Validation runs every `val_interval` epochs (default 1). Running it every epoch catches the best model sooner but costs time; for expensive training runs, a larger `val_interval` can be chosen.

---

## 9. Handling Class Imbalance in the Metric

---

### 9.1 Why overall voxel accuracy is misleading

As with the dataset imbalance problem in the loss function, evaluation accuracy on a tumor segmentation model can be dominated by the background class. A model that never predicts any tumor at all would score over 95% voxel accuracy.

The solution is the same: compute metrics separately per class and average.

### 9.2 The Dice Metric

MONAI's `DiceMetric` computes the Dice coefficient per class and returns the mean:

```
dice_metric = DiceMetric(include_background=False, reduction="mean")
```

`include_background=False` excludes class 0 (background) from the per-class average. Including it would again inflate the score artificially, since background is easy to predict correctly.

The metric operates on one-hot tensors, which is why the post-processing transform converts logits to one-hot predictions:

```
post_trans = Compose([
    Activations(softmax=True),
    AsDiscrete(argmax=True, to_onehot=4)
])
```

---

## **Part IV. Inference and Visualization**

## 10. Sliding-Window Inference

---

### 10.1 The memory problem

A whole-volume BraTS scan at 1mm isotropic resolution, after preprocessing, is roughly (4, 240, 240, 155). A single forward pass through the 3D U-Net on this volume would require allocating all the intermediate feature maps at their native resolution: at the input resolution, the bottleneck feature map alone is (B, 128, 30, 30, 19), and the input has not been reduced. The full computation would require several gigabytes of GPU memory per sample, more than is available on most hardware.

Patch-based training avoids this by never passing the full volume. But evaluation on patches introduces the randomness problem: which patches happen to be sampled affects the score.

### 10.2 Sliding-window inference

The solution is sliding-window inference: divide the full volume into overlapping patches, run inference on each patch, and stitch the predictions back together by averaging the softmax outputs in the overlapping regions:

```
from monai.inferers import sliding_window_inference

with torch.no_grad():
    val_output = sliding_window_inference(
        inputs=val_image,
        roi_size=(96, 96, 96),
        sw_batch_size=2,
        predictor=model,
        overlap=0.5,
    )
```

`overlap=0.5` means adjacent windows share half their extent in each dimension. The overlapping regions produce two (or more) predictions per voxel, which are averaged before `argmax`. Averaging reduces boundary artifacts that can appear when each window's predictions are taken independently.

The GPU sees only the current batch of patches at any one time. The memory cost is bounded by the patch size, not the full volume.

### 10.3 Post-processing

After sliding-window inference, the raw output has shape (B, 4, H, W, D): one logit per class per voxel. Converting to a discrete class label requires:

```
pred_mask = torch.argmax(val_output, dim=1)[0].cpu().numpy()
# pred_mask.shape: (H, W, D)
```

`argmax` along the class dimension picks the highest-confidence prediction for each voxel.

---

## 11. The BraTS Evaluation Hierarchy

---

### 11.1 Why not just Dice per label

The raw label values (0, 1, 2, 3) in BraTS do not correspond directly to the clinical regions of interest. The standard BraTS evaluation measures Dice on three overlapping *derived* regions, not on the individual labels:

| Region          | Abbreviation | Voxel Labels Included     |
|-----------------|--------------|---------------------------|
| Whole Tumor     | WT           | 1 (NCR) + 2 (ED) + 3 (ET) |
| Tumor Core      | TC           | 1 (NCR) + 3 (ET)          |
| Enhancing Tumor | ET           | 3 only                    |

The logic is clinical. A surgeon planning a resection needs to know the full tumor extent (WT) and the metabolically active core (TC). A radiation oncologist targeting the enhancing lesion specifically needs ET. Each of these is separately meaningful and separately scored.

### 11.2 Computing the metrics

```
def compute_brats_metrics(gt_mask, pred_mask, voxel_spacing=None):
    # Whole Tumor: all labels > 0
    gt_wt = (gt_mask >= 1)
    pred_wt = (pred_mask >= 1)

    # Tumor Core: labels 1 and 3
    gt_tc = np.isin(gt_mask, [1, 3])
    pred_tc = np.isin(pred_mask, [1, 3])

    # Enhancing Tumor: label 3 only
    gt_et = (gt_mask == 3)
    pred_et = (pred_mask == 3)

    # Dice for each
    ...
    metrics["Mean_Dice"] = (metrics["WT_Dice"] + metrics["TC_Dice"]
                           + metrics["ET_Dice"]) / 3.0
```

The three Dice scores are averaged to a single Mean Dice, which is the headline number in BraTS leaderboards.

---



## 12. Publication-Quality Static Visualization

---

### 12.1 Why static

The notebooks in this project do not use interactive sliders. Interactive widgets (ipywidgets, Plotly) require a running kernel to work; they render as blank boxes in any static context: a PDF, a GitHub preview, an nbconvert output. Every interactive figure in the original design was a figure that disappeared the moment the kernel was closed.

Static, publication-quality figures solve this completely. They are saved as PNG files and embedded in the notebook as output. They look correct in every rendering context.

### 12.2 Multi-Planar Reconstruction (MPR)

The standard radiological view for a 3D volume is to show three orthogonal cross-sections simultaneously: Sagittal (fixed X slice), Coronal (fixed Y slice), and Axial (fixed Z slice). Together they give a complete spatial impression of a 3D structure that no single 2D view can convey.

The `plot_orthogonal_slices` function centers the three views automatically on the tumor centroid if a mask is provided, annotates anatomical orientation markers on each view (S/I, A/P, L/R), draws synchronized crosshairs, and reports physical coordinates in mm alongside voxel indices:

```
fig = plot_orthogonal_slices(  
    volume=img_data,  
    mask=lbl_data,  
    affine=affine_matrix,  
    title=f"Multi-Planar Orthogonal Reconstruction - {subject}",  
    show_crosshairs=True,  
    show_labels=True  
)
```

### 12.3 Volumetric Slice Montage

A single MPR at one location answers "what does the tumor look like here?" but not "where does the tumor appear across the brain depth?". The volumetric montage answers the second question: it selects a set of evenly-spaced axial slices spanning the tumor's full extent and arranges them in a grid:

```
fig = plot_volumetric_montage(  
    volume=img_data,  
    mask=lbl_data,  
    affine=affine_matrix,  
    num_slices=10,  
    title="Axial Slice Montage Across Tumor Depth"  
)
```

Slices containing tumor are annotated with a star marker and displayed with gold title text. Slices without tumor are annotated in gray. This makes it immediately visible which slices are clinically relevant.

### 12.4 The Affine Coordinate Space Diagram

The affine chapter (Chapter 3) is mathematical. The visualization makes it concrete: a 3D scatter plot of the physical bounding box of the volume, with the origin and center marked, the three scanner axes (R, A, S) drawn as colored arrows, and a text card showing the complete affine matrix and a table of landmark coordinates:

```
fig = plot_affine_coordinate_space(
    affine=affine_matrix,
    shape=img_data.shape,
    mask=lbl_data,
    title="Affine Transformation & Physical Scanner Coordinate Space"
)
```

The tumor centroid is marked in the 3D plot if a mask is provided, showing where the pathology sits in physical space.

## 12.5 Static 3D Tumor Surface Mesh

The 3D shape of the tumor is reconstructed using the Marching Cubes algorithm, which extracts a triangle mesh from a binary volumetric mask at a specified isosurface level. The three tumor sub-regions (NCR, ED, ET) are rendered as separate surfaces with different colors and transparencies.

Because an interactive Plotly figure requires WebGL and a running kernel, the static version uses Matplotlib's Poly3DCollection and renders the same mesh from four canonical anatomical viewpoints simultaneously:

```
fig = plot_3d_tumor_mesh_static(
    mask=lbl_data,
    affine=affine_matrix,
    step_size=2,
    title="3D Brain Tumor Surface Reconstruction (4-View)"
)
```

The four views are: Isometric perspective (elev=25, azimuth=45), Superior/Axial (top-down), Anterior/Coronal (front), and Lateral/Sagittal (side). Each shows the same geometry from a different angle, giving the reader a complete 3D impression from a 2D figure.

## 12.6 The Segmentation Evaluation Dashboard

After running inference, the evaluation dashboard provides a multi-slice comparison matrix across the tumor's axial extent. For each of four key slices, it shows four panels side-by-side:

1. **MRI Structural Scan** (T1ce channel)
2. **Expert Ground Truth Annotation**
3. **3D U-Net Model Prediction**
4. **Spatial Error Map**: True Positives in green, False Positives in red, False Negatives in blue

```
fig = plot_segmentation_evaluation_dashboard(
    image_4d=val_img_np,
    gt_mask=val_label,
    pred_mask=pred_mask,
    metrics=compute_brats_metrics(val_label, pred_mask),
    title="3D U-Net Volumetric Segmentation & Spatial Error Matrix"
)
```

The slice-level Dice score is annotated on each row. The color of the Dice score annotation changes from green ( $>0.8$ ) to yellow ( $>0.5$ ) to red ( $<0.5$ ). Overall BraTS WT/TC/ET Dice scores from `compute_brats_metrics` appear in the figure title.

The spatial error map is the most diagnostic panel. Red pixels (FP, over-segmentation) indicate where the model predicted tumor that was not there; blue pixels (FN, under-segmentation) indicate tumor the model missed. The spatial pattern of errors tells more than a single number: a model that misses the tumor core will show blue in the center; one that over-segments the edema boundary will show red at the periphery.

---

## 13. Maximum Intensity Projection

---

Maximum Intensity Projection (MIP) is a standard radiological technique for visualizing 3D volumes without a 3D renderer. For each ray through the volume along a given axis, the maximum voxel intensity along that ray is projected onto the 2D output image:

```
sag_mip = np.rot90(np.max(vol_3d, axis=0))    # Sagittal
cor_mip = np.rot90(np.max(vol_3d, axis=1))    # Coronal
ax_mip  = np.rot90(np.max(vol_3d, axis=2))    # Axial
```

MIP is particularly effective for visualizing vascular structures (where bright contrast enhances the entire vessel tree) and for showing the full 3D extent of a bright lesion in a single 2D image. The tumor, being bright in T1ce, appears as a bright projection that reveals its 3D shape without needing rotation.

The `plot_triplanar_mip` function produces all three projections simultaneously, with the tumor mask projected as a color overlay if provided.

---

## 14. Where This Leads

---

The machinery built in this project maps directly to harder 3D medical problems and to the adjacent field of diffusion MRI tractography.

In tractography, nerve fiber pathways through the brain are reconstructed from diffusion MRI as three-dimensional streamlines: sequences of 3D points tracing the path of each fiber bundle. The spatial coordinate system for those streamlines is exactly the physical space defined by the NIfTI affine matrix. A fiber at a given physical location in mm can be mapped to voxel indices in the segmentation volume and vice versa, using the same voxel-to-world and world-to-voxel functions developed in Chapter 3.

The volumetric segmentation model from this project can provide anatomical context for fiber classification: a fiber running through voxels labeled as white matter has a different biological interpretation than one running through voxels labeled as tumor or edema.

Conversely, fiber classification architectures (EdgeConv, sequence-aware Graph Neural Networks) operate on the same 3D coordinate space as volumetric models but treat the data as a set of curves rather than a dense grid. The connection between dense volumetric representations and sparse geometric representations is the central concept bridging this project to tractography.

---

## Appendix A. Environment and Reproduction

---

**Software.** Python 3.14, torch 2.14.0, monai 1.3+, nibabel 5.0+, scikit-image 0.26+, matplotlib 3.7+. Full dependencies are in `requirements.txt`.

**Data.** The MSD Task01\_BrainTumour dataset is downloaded automatically by `download_data.py`, which uses MONAI's `download_and_extract` with MD5 verification:

```
python download_data.py
```

This downloads approximately 3 GB from the Medical Segmentation Decathlon S3 bucket. After extraction, the dataset is organized as:

```
data/Task01_BrainTumour/  
  dataset.json  
  imagesTr/BRATS_001.nii.gz ...  
  labelsTr/BRATS_001.nii.gz ...
```

**Order of execution.** Run `build_notebooks.py` once to generate the Jupyter notebooks, then open them in order: `01_nifti_and_affine_exploration.ipynb` first, `02_monai_3d_segmentation.ipynb` second. Training saves to `results/best_metric_model.pth`; the segmentation notebook loads from that path if a checkpoint already exists.

**Command-line training.** The full training loop is also available without Jupyter:

```
python -m src.train --epochs 20 --batch_size 2 --lr 0.0002
```

---

## Appendix B. Pitfalls Encountered

---

Recorded because each one cost real time and each is the kind of thing that does not appear in documentation.

**Interactive widget output disappears without a running kernel.** The original notebook design used ipywidgets sliders and Plotly for all visualization. Every such figure is blank in any static rendering context: nbconvert HTML, nbviewer, GitHub notebook preview, and PDF. All visualization was replaced with static Matplotlib figures that are saved to disk and embedded as PNG cell outputs. The rule is: if a figure cannot be opened in a browser without a running kernel, it is not suitable for documentation.

**tight\_layout fails silently for 3D Matplotlib axes.** Calling `plt.tight_layout()` on a figure that contains `projection='3d'` subplots emits a `UserWarning: Tight layout not applied` and leaves the layout unchanged. The fix is to use `fig.subplots_adjust()` or `bbox_inches='tight'` in `savefig` instead, which applies the adjustment at render time rather than at figure construction time.

**NormalizeIntensityd with nonzero=False inflates the normalization baseline.** The zero background in brain MRI is not zero because the brain has zero signal there; it is zero because no measurement was made. Including it in the normalization statistics shifts the estimated mean downward, making all brain tissue appear artificially bright after normalization. Always use `nonzero=True`.

**Nearest-neighbor interpolation is required for label resampling.** `Spacingd` resamples both image and label to the target resolution. Using bilinear interpolation on integer-valued class labels produces fractional values (e.g. a boundary voxel between label 1 and label 2 becomes 1.7). These fractional values are then rounded in unpredictable ways. Always pass `mode=("bilinear", "nearest")` to apply bilinear to the image and nearest-neighbor to the label.

**`torch.save(model.state_dict(), path)` saves a reference, not a copy.** If the model's weights continue to be updated by training steps after the save call, the saved checkpoint silently becomes the current weights, not the best ones. Use `.detach().cpu().clone()` before storing any tensor you want to keep constant.

**`global_max_pool` needs the batch vector.** Without the batch vector, a pooling step that is meant to collapse "many voxels" into "one prediction per volume" would pool across volume boundaries and mix up different patients' predictions. The batch vector produced by the `DataLoader` keeps each volume's voxels associated with the correct sample index.

---

## Appendix C. Glossary

| Term                   | One-line definition                                                                       |
|------------------------|-------------------------------------------------------------------------------------------|
| Affine matrix          | 4x4 matrix mapping voxel indices to physical scanner coordinates in mm                    |
| Augmentation           | Random transformations applied during training to increase effective dataset size         |
| BraTS                  | Brain Tumor Segmentation challenge; the source of the dataset used here                   |
| Channel-first          | Array layout where the channel axis comes before spatial axes: (C, D, H, W)               |
| Cosine annealing       | Learning rate schedule that decays smoothly from initial value to near zero               |
| CosineAnnealingLR      | PyTorch scheduler implementing cosine annealing                                           |
| DiceCELoss             | Combined Dice and cross-entropy loss for medical segmentation                             |
| Dice coefficient       | Overlap metric: $2 * \text{intersection} / (\text{pred} + \text{gt})$ , ranges 0 to 1     |
| Dictionary transform   | MONAI transform operating on a dict of named arrays rather than a single array            |
| Edema (ED)             | Peritumoral edema: fluid swelling surrounding the tumor, BraTS label 2                    |
| Enhancing Tumor (ET)   | GD-enhancing active tumor core, BraTS label 3                                             |
| Field of View (FOV)    | Physical extent of the scanned volume in mm                                               |
| FLAIR                  | MRI acquisition suppressing CSF signal; highlights peritumoral edema                      |
| Gadolinium             | Contrast agent used in T1ce MRI; leaks into tissue where blood-brain barrier is disrupted |
| Instance Normalization | Normalization per sample, per channel; stable with small batch sizes                      |
| Isotropic              | Having equal voxel spacing in all three spatial dimensions                                |
| MaxPool3d              | 3D max pooling; reduces spatial resolution by taking the maximum in each window           |
| Marching Cubes         | Algorithm to extract a triangle mesh from a volumetric scalar field                       |
| MIP                    | Maximum Intensity Projection: projects maximum voxel value along a ray onto 2D            |
| MSD                    | Medical Segmentation Decathlon; source of the Task01_BrainTumour dataset                  |
| Necrotic Core (NCR)    | Dead tissue in the tumor center, BraTS label 1                                            |
| NIFTI                  | Neuroimaging Informatics Technology Initiative; standard format for neuroimaging data     |
| Normalization          | Rescaling intensities to zero mean and unit standard deviation per channel                |

| Term                     | One-line definition                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------|
| Orientation              | Which anatomical direction each array axis points: RAS, LPS, etc.                          |
| Patch                    | A small 3D sub-volume extracted from a larger volume for training                          |
| RAS                      | Right-Anterior-Superior anatomical orientation convention                                  |
| Resampling               | Interpolating a volume to a different spatial resolution                                   |
| Segmentation             | Assigning a class label to every voxel in an image                                         |
| Skip connection          | Path copying encoder feature maps directly to the corresponding decoder level              |
| Sliding-window inference | Dividing a large volume into overlapping patches for inference, then stitching predictions |
| Softmax                  | Function normalizing a vector of logits to probabilities that sum to 1                     |
| Spacing                  | Physical voxel size in mm per dimension                                                    |
| T1ce                     | T1-weighted contrast-enhanced MRI; marks active tumor core                                 |
| T1w                      | T1-weighted native MRI; standard anatomical contrast                                       |
| T2w                      | T2-weighted MRI; sensitive to fluid; marks broad tumor extent                              |
| Transposed convolution   | Learned upsampling operation; inverse of convolution in terms of spatial resolution        |
| Tumor Core (TC)          | BraTS sub-region: NCR + ET (labels 1 + 3)                                                  |
| U-Net                    | Encoder-decoder architecture with skip connections for dense prediction                    |
| Voxel                    | One element of a 3D volume array; the 3D equivalent of a pixel                             |
| Whole Tumor (WT)         | BraTS sub-region: all tumor labels (1 + 2 + 3)                                             |
| Zero-pad                 | Padding convolution inputs with zeros to maintain spatial dimensions                       |